

AI AURUM

Intelligent Assistant Platform

Full Project Build Summary

Generated July 02, 2026

Yuvan Industries • 104 Tasks Completed

104	Tasks Completed	20+	API Routes	11	Specialist Agents	5	Memory Layers	30+	Tools & Plugins
-----	-----------------	-----	------------	----	-------------------	---	---------------	-----	-----------------

1. Project Overview

AI Aurum is a full-stack intelligent assistant platform built with Flask (Python). It features a streaming chat interface, multi-agent AI orchestration, autonomous task planning, a dynamic plugin system, voice capabilities, and a rich set of specialist tools. The platform was built incrementally across 104 tasks, evolving from a simple chatbot into a production-grade AI operating system.

The project is owned by **Yuvan Industries** and uses GitHub Models (Azure) as the primary AI backend, with local Ollama as fallback. The codebase uses Flask Blueprints, SQLite for persistence, and ChromaDB for vector memory.

Technology Stack

Layer	Technology
Backend	Python 3.x + Flask + Blueprints
AI Models	GPT-4o, GPT-4o-mini, o1-mini via GitHub Models (Azure)
Local AI	Ollama (llama3.2, mistral, phi3)
Database	SQLite (users, chats, memory, tools, analytics)
Vector Memory	ChromaDB (semantic search)
Scheduler	APScheduler (background jobs)
Frontend	Vanilla JS + SSE streaming + PWA
Voice	Whisper STT + ElevenLabs / edge-tts TTS
Browser Agent	Playwright
Auth	Session-based + Flask-Limiter + CSRF

2. System Architecture

The application follows a layered Blueprint architecture. Each domain is a separate Flask Blueprint registered in **app.py**. Services are injected via a **_deps** dictionary pattern, keeping routes decoupled from business logic.

app.py

20 blueprints registered; Flask-Limiter; session age enforcement; APScheduler

routes/

20 route files: auth, chat, stream, research, upload, tools, files, settings, admin, debate, agents, planning, workflow, workspace, dashboard, analytics, vote, voice, trace, subagent

services/

18 service modules: auth, ai, speech, chat, event_bus, capability_registry, agent_health, memory_layers, sandbox, tracer, voice, auto_learn, task_queue, model_voting, ollama, personal_twin, error_handler

agents/

11 specialist agents: CEO, Planner, Researcher, Programmer, Debugger, Reviewer, Memory Manager, Vision, Voice, Automation, Browser, Security

tools/

30+ tools: calculator, weather, web_search, code_runner, document tools, electrical_engineering, meeting_assistant, knowledge_graph, system_monitor, dev_agent, and more

planning/

Autonomous planner + SSE executor (planner.py, executor.py)

workflows/

Workflow engine with trigger/tool/agent/condition/wait steps

plugins/

Auto-discovery: drop .py file with NAME/DESCRIPTION/run() to add plugin

assistant/

Refactored monolith split into 9 modules for maintainability

7 Architectural Layers (Tasks #97-104)

#	Component	Description
97	Event Bus	Thread-safe pub/sub; wildcard subscriptions; async dispatch; history ring
98	Capability Registry	Keyword+skill scoring to rank agents for any task; replaces hardcoded routing
99	Agent Health Monitor	SQLite rolling metrics; health score = success*0.6 - latency*0.25 - timeout*0.15
100	Confidence Scores	Fast gpt-4o-mini rates every agent response 0-1 for CEO decision-making
101	Memory Layers (5-tier)	Working (TTL) + Conversation (SQLite) + KnowledgeGraph + Vector (ChromaDB) + Archive (FTS5)
102	Tool Sandboxing	PluginPermissions manifest; threading timeout (fast) vs subprocess isolation (strong)
103	Observability/Tracing	trace_id per request; spans for every sub-operation; DAG graph via /traces API

3. Feature Roadmap — Completed

P1 — Core Intelligence

Feature	What it does
Multi-Agent AI Team (#79)	CEO orchestrator routes goals to 11 specialist agents via JSON plan. Parallel execution with ThreadPools.
Autonomous Task Planning (#80)	AI decomposes goals into PlanStep DAG. Respects dependencies. SSE-streams progress. Post-execution summary.
Dynamic Plugin System (#81)	Auto-discovers *.py files in plugins/. Requires only NAME, DESCRIPTION, run(). Zero-config registration.

P2 — Extended Capabilities

Feature	What it does
AI Workflow Builder (#82)	Trigger/tool/agent/condition/wait steps. SQLite-persisted. AI generates workflows from natural language.
Enhanced Browser Agent (#83)	Playwright-backed. Full navigation, form-fill, click, screenshot, scrape. Structured JSON results.
Document Intelligence (#84)	Unified agent for PDF/DOCX/PPTX/Excel/OCR. Extracts, summarises, and answers questions about documents.
Voice Assistant (#85)	Whisper STT + ElevenLabs/edge-tts TTS. Wake word detection. Streaming voice I/O via /voice/* routes.

P3 — Workspace & Productivity

Feature	What it does
Workspace / Projects (#86)	Per-user projects with name, description, tech stack, goals, file list, memory. Persistent context injected.
Autonomous Coding Team (#87)	dev_agent tool: CEO delegates to programmer + reviewer + debugger. Multi-file project builder with git integration.
AI OS Dashboard (#88)	Live SSE-streamed metrics: CPU, RAM, tokens, agents, tasks, files, plugins, analytics.

P4 — Intelligence Upgrades

Feature	What it does
Model Voting (#89)	Query 3+ models in parallel. Majority-vote consensus. Confidence weighting. SSE stream of individual votes.
Automatic Learning (#90)	Daily APScheduler job extracts lessons from recent conversations. Stores as structured knowledge in SQLite.
Performance Analytics (#91)	Tracks response time, model latency, tool success/fail rate, token usage, user satisfaction per model.

P5 — Specialist Domains

Feature	What it does
Electrical Engineering (#92)	IEC 60364/60076/60909/62446 calculations: cable sizing, transformer sizing, fault level, solar design, BESS.
Engineering Drawing Reader (#93)	GPT-4o vision analyses P&IDs, circuit diagrams, floor plans. Extracts components, identifies issues, suggests fixes.
AI Meeting Assistant (#94)	Whisper transcription + summarise + action items + formal minutes + follow-up email draft. Full pipeline.
Personal AI Twin (#95)	Learns writing style, vocabulary, tone from user samples. Generates content that sounds like the user.

4. Complete Task Log (104 Tasks)

Foundation (1-20)

#	Task	Status
1	Create vector_memory.py (ChromaDB semantic memory)	Done
2	Add model routing to smith_web.py	Done
3	Add personality settings (DB + API + UI)	Done
4	Wire vector memory into /stream endpoint	Done
5	Verify: test all features run without errors	Done
6	Create tools/ plugin system with registry	Done
7	Build 6 tools: calculator, weather, news, email, calendar, reminders	Done
8	Add /tools routes to smith_web.py	Done
9	Add Tools UI panel to index.html	Done
10	Verify syntax + integration	Done
11	Create tools/code_runner.py + tools/git_tool.py	Done
12	Add /code /files /git routes to smith_web.py	Done
13	Add Coding Panel to index.html	Done
14	Verify all files syntax-clean	Done
15	Add OpenAI function-calling tool loop to tools/__init__.py	Done
16	Create agent.py — planner + ReAct tool-calling loop	Done
17	Migrate assistant.py JSON memory to SQLite	Done
18	Refactor /stream into routes/stream_routes.py Blueprint	Done
19	Wire agent loop into /stream + verify all syntax	Done
20	Update README.md for AssistNeo	Done

Fixes & Security (21-50)

#	Task	Status
21	Fix calculator username arg error + 401 friendly message	Done
22	Fix security issues and bugs	Done
23	Create Internet tools (web search, browse, YouTube, deep research)	Done
24	Create Document tools (PDF, DOCX, PPT, Excel, OCR, PDF analysis)	Done
25	Add Save & Push to GitHub Desktop button	Done
26	Make UI mobile-responsive with PWA support	Done
27	Create vision_tool.py — image analysis via GPT-4o vision	Done
28	Add multimodal support to agent.py	Done
29	Add /upload/image route to smith_web.py	Done
30	Add camera + image upload UI to index.html	Done

31	Build task scheduler (APScheduler)	Done
32	Build browser automation tool (Playwright)	Done
33	Build workflow / task chain tool	Done
34	Create app.py — slim Flask entry point	Done
35	Create services/ layer	Done
36	Split routes into blueprints	Done
37	Verify refactor — all routes work, smith_web.py replaced	Done
38	Add structured logging to all files	Done
39	Async tool execution — run independent tools concurrently	Done
40	Smarter memory retrieval — threshold, recency, importance	Done
41	Better tool selection — failure history, cost, runtime tracking	Done
42	Full syntax + feature verification	Done
43	Create tools/tool_metrics.py — per-tool SQLite metrics	Done
44	Integrate tool_metrics into tools/__init__.py call()	Done
45	Full syntax + feature verification sweep	Done
46	Create tests/ suite (pytest, API, tools, memory, agent)	Done
47	Create config/ layer (base, development, production)	Done
48	Add richer permission model (roles + admin decorator)	Done
49	Centralized error handling (error_handler.py + Flask handlers)	Done
50	Final verification sweep	Done

Intelligence & UX (51-78)

#	Task	Status
51	Rename AI from Neo/AssistNeo to AI Aurum throughout codebase	Done
52	Increase AI response speed	Done
53	Add Guest User login with limited features	Done
54	Make logo bigger in UI	Done
55	Fix /code/run security — restrict to admin	Done
56	Fix /tools/custom/save — replace string blacklist with AST validation	Done
57	Fix assistant.py eval() — replace with AST-safe eval	Done
58	Git hygiene — untrack binaries/DB/logs	Done
59	Persistent Memory — project context injection	Done
60	Self-Evolving Skills — DB + tool + UI	Done
61	Parallel Sub-Agents — orchestrator + endpoint + UI	Done
62	MCP Client tool — connect to external MCP servers	Done
63	Upgrade web_search — Gemini grounding + news/research/price/compare modes	Done

64	Create system_monitor tool — CPU/RAM/GPU/temp + threshold alerts	Done
65	Upgrade youtube tool — add summarize/get_info/trending actions	Done
66	Create dev_agent tool — autonomous multi-file project builder	Done
67	Create flight_finder tool — search flights via Gemini parsing	Done
68	Create messaging tool — Telegram Bot API + Discord webhook	Done
69	Verify all new tools syntax-clean	Done
70	Split assistant.py into assistant/ package	Done
71	Upgrade capability router — scoring-based model selection	Done
72	Add self-evaluation after every AI response	Done
73	Build multi-agent debate mode	Done
74	Auth hardening — rate limiting + session expiry + CSRF	Done
75	Add RQ background task queue	Done
76	Create knowledge graph tool	Done
77	Add local model fallback — Ollama service	Done
78	Verify all new files syntax-clean + update app.py	Done

Feature Roadmap P1-P5 (79-95)

#	Task	Status
79	P1: Build Multi-Agent AI Team (agents/ package)	Done
80	P1: Autonomous Task Planning engine	Done
81	P1: Dynamic Plugin System (auto-discovery)	Done
82	P2: AI Workflow Builder (routes + persistence)	Done
83	P2: Enhanced Browser Agent	Done
84	P2: Document Intelligence (unified doc agent)	Done
85	P2: Voice Assistant (streaming STT + wake word)	Done
86	P3: Workspace system (Projects with files/memory/goals)	Done
87	P3: Autonomous Coding Team pipeline	Done
88	P3: AI OS Dashboard (live metrics)	Done
89	P4: Model Voting (multi-model consensus)	Done
90	P4: Automatic Learning (daily lesson extraction)	Done
91	P4: Performance Analytics (metrics dashboard)	Done
92	P5: Electrical Engineering Assistant tool	Done
93	P5: Engineering Drawing Reader (vision tool)	Done
94	P5: AI Meeting Assistant	Done
95	P5: Personal AI Twin (style learning)	Done

Architecture & Wiring (96-104)

#	Task	Status
96	Wire all new blueprints into app.py + syntax verify	Done
97	Arch: Event Bus (pub/sub decoupling)	Done
98	Arch: Capability Registry (skill-based agent routing)	Done
99	Arch: Agent Health Monitoring	Done
100	Arch: Confidence Scores on every agent response	Done
101	Arch: Memory Layers (5-tier system)	Done
102	Arch: Tool Sandboxing (plugin isolation)	Done
103	Arch: Observability/Tracing (execution graph)	Done
104	Wire arch components into agents + app.py	Done

5. Key Files & APIs Reference

Critical Files

File	Purpose
<code>app.py</code>	Flask entry point — registers 20 blueprints, limiter, scheduler
<code>agents/base_agent.py</code>	BaseAgent: <code>think()</code> , <code>scored_think()</code> , health metrics, event bus
<code>agents/ceo_agent.py</code>	CEO orchestrator — uses CapabilityRegistry for data-driven routing
<code>agents/__init__.py</code>	<code>run_team()</code> — parallel agent execution + synthesis
<code>planning/planner.py</code>	<code>create_plan()</code> + <code>execute_plan()</code> — autonomous multi-step planning
<code>planning/executor.py</code>	<code>stream_execute()</code> — SSE generator for plan progress
<code>services/event_bus.py</code>	Singleton EventBus — pub/sub with wildcards, async dispatch
<code>services/capability_registry.py</code>	CapabilityRegistry — keyword+skill scoring for agent ranking
<code>services/agent_health.py</code>	AgentHealthMonitor — rolling SQLite metrics + health score
<code>services/memory_layers.py</code>	MemorySystem — 5-tier recall + <code>context_string()</code> injection
<code>services/sandbox.py</code>	Sandbox — threading timeout vs subprocess plugin isolation
<code>services/tracer.py</code>	Tracer — <code>trace_id</code> , spans, DAG graph, SQLite storage
<code>routes/stream_routes.py</code>	<code>/stream</code> SSE — model routing, memory injection, self-eval, tracing
<code>routes/agents_routes.py</code>	<code>/agents</code> , <code>/agents/health</code> , <code>/agents/best</code> , <code>/agents/<n>/ask</code>
<code>routes/trace_routes.py</code>	<code>/traces</code> , <code>/traces/<id></code> , <code>/traces/<id>/graph</code>
<code>tools/electrical_engineering.py</code>	IEC cable, transformer, fault level, solar, BESS, tariff calcs
<code>tools/meeting_assistant.py</code>	Transcribe + summarise + action items + minutes + email draft
<code>workflows/engine.py</code>	Workflow engine — trigger/tool/agent/condition/wait steps
<code>plugins/__init__.py</code>	Auto-discovery — scan <code>plugins/*.py</code> , register NAME/DESCRIPTION/run()
<code>vector_memory.py</code>	ChromaDB semantic memory — store + retrieve relevant past chats
<code>db.py</code>	SQLite helpers — users, chats, memories, skills, analytics, projects

API Endpoints

Route	Method	Description
<code>/stream</code>	POST	SSE streaming chat with tool use, memory, self-eval, tracing
<code>/agents/run</code>	POST	CEO multi-agent team — SSE streamed
<code>/agents/health</code>	GET	Health scores for all 11 agents
<code>/agents/best</code>	POST	Best agent for a given task (registry + health)
<code>/plan</code>	POST	Autonomous planning + execution — SSE streamed
<code>/debate</code>	POST	Multi-model debate on a topic — SSE streamed
<code>/vote</code>	POST	Model voting consensus — SSE streamed
<code>/workflows</code>	POST	Create/run AI workflows

/traces	GET	List recent request traces
/traces/<id>/graph	GET	Execution DAG for a specific trace
/voice/transcribe	POST	Whisper speech-to-text
/voice/synthesise	POST	TTS audio generation
/dashboard/stream	GET	Live system metrics SSE stream
/analytics	GET	Performance analytics dashboard
/tools/run	POST	Execute any registered tool
/research	POST	Deep web research mode
/memory	GET	User memory facts
/workspace/projects	GET	List user projects
/admin/metrics	GET	Admin system metrics (admin only)

6. Environment & Setup

Required Environment Variables

Variable	Purpose	Required
GITHUB_TOKEN	GitHub Models (Azure) API key — primary AI backend	Yes
FLASK_SECRET_KEY	Session encryption key	Yes
ELEVENLABS_API_KEY	ElevenLabs TTS voice synthesis	Optional
OPENAI_API_KEY	OpenAI direct API (fallback)	Optional
GEMINI_API_KEY	Google Gemini (web search grounding)	Optional
TELEGRAM_TOKEN	Telegram bot messaging	Optional
DISCORD_WEBHOOK	Discord webhook messaging	Optional
WORKSPACE_DIR	Custom workspace folder path	Optional
PORT	Flask port (default: 5000)	Optional
FLASK_DEBUG	Set to 1 for debug mode	Optional
CODE_MODEL	Model for code tasks (default: gpt-4o)	Optional
FAST_MODEL	Model for fast tasks (default: gpt-4o-mini)	Optional
VISION_MODEL	Model for vision tasks (default: gpt-4o)	Optional
REASON_MODEL	Model for reasoning tasks (default: o1-mini)	Optional

Quick Start

Step 1: Install dependencies

```
pip install -r requirements.txt
```

Step 2: Set environment variables

```
Copy .env.example to .env and fill in GITHUB_TOKEN
```

Step 3: Run the app

```
python app.py
```

Step 4: Open in browser

```
http://localhost:5000
```

Step 5: Push to GitHub

```
Run push.py (auto-clears index.lock if stuck)
```

7. Notable Bugs Fixed During Build

Bug	Cause & Fix
Windows path truncation	Edit/Write tools silently truncated files when path contained spaces/parentheses. Fix: always use python3 open
app.py SyntaxError	Multi-line Flask-Limiter decorator chain caused "(" was never closed". Fix: simplified to single Limiter() call.
stream_routes KeyError	model_name dict only had 3 keys; "vision" and "reason" caused KeyError. Fix: added all 5 keys + .get() fallback
base_agent.py truncated	File truncated at line 70 mid-word. Fix: rewrote via Python subprocess.
meeting_assistant.py	Em-dash (U+2014) inside f-strings caused SyntaxError. Fix: replaced with hyphen.

addActions infinite loop	JS function hoisting meant patched addActions called itself via _origAddActions. Fix: merged into single definition
JS catch block corruption	Two overlapping catch blocks merged into one line, breaking all JavaScript on the page (Enter key, send button)
git index.lock	Stale lock file blocked all git operations. Fix: push.py now auto-removes lock file at startup.